1 Description:

My plan for my project, is a scene-referenced HDR render of a Mandelblub fractal utilizing applications and techniques such as a raymarching compute shader pass and tone mapping for a final scene-referred HDR image in LDR image format (sRGB). Raymarching is a technique in which you utilize the shader's power of parallelization to march a ray into your scene for every pixel and see what you hit as the ray travels which colors the pixel accordingly. This is done using SDF's or signed distance functions which are functions that, for any point in space, returns the shortest distance to a surface, with the sign indicating whether the point is inside, outside, or exactly on that surface. This technique is done per fragment, so it can be pretty computationally heavy depending on factors like iteration steps for your ray. You can also utilize this technique to also create hard or soft shadows pretty easily as once you know where a ray has intersected something in your scene you can then march a ray from the intersection point to your scene's light to see if there is any occluding objects. I plan to utilize a compute shader for this so that I can easily write to an HDR 16-bit floating-point formatted storage image, so I can then get a more vibrant scene in my fractal where I will be coloring using orbit trap coloring, Phong illumination and Blinn-Phong lighting. Orbit trap coloring is the ability to color what you hit with your ray as you march it based on the distance the ray has traveled at the intersection. I will be using this as my base material color for then a Blinn-Phong lighting calculation (with no clamp) to produce color values greater than the normal zero-to-one range which I will then output to the HDR 16-bit float formatted storage image that I will then process with tone mapping. Tone mapping is the process of taking all the color values of every pixel and ensuring that all pixel color values end up back into LDR range (0 - 1.0). This is done by using a nonlinear function to each HDR color value so that bright values get compressed not clipped, mid-tones stay balanced, and shadows retain detail instead of being black. This will produce a final image that is then LDR format ready and therefore ready to be displayed on the device.

2 Application:

The techniques I plan to implement have a wide variety of usage. Raymarching is a powerful technique to render surfaces that otherwise would be hard to model using hard geometry such as triangles. Precisely why I want to render a Mandelblub, it demonstrates the power to procedurally render shapes that have no closed-form mesh and objects that otherwise wouldn't be able to be done. As mentioned above too there are numerous other advantages that raymarching offers due to the utilization of SDF's such as the low overhead to setup soft or hard shadows, CSG or constructive solid geometry operations (union, subtract, intersect) with option of continuous blending (smooth, rounded, or morphing), easy instancing, and repetition. Normally raymarching is done by sending a full screen triangle or quad to the vertex shader whose output then gets rasterized

and ready for modification in the fragment shader. This provides a "canvas" or direct control over each pixel. However, in my implementation I will be utilizing a compute shader which allows for the control of global and local workgroups so that you can present your parallel problem/technique (in this case raymarching) to your GPU. Compute shader pipelines are really useful to compute problems or techniques that require massive parallelization such as in the case of calculating each pixel's HDR value for out display screen. Compute shaders eliminate the overhead of the fullscreen triangle or quad as we can write out the results of the compute shader to a storage image directly. This allows for cool post processing effects to be done in other passes or you can access the results back in the fragment shader. This brings up the HDR rendering portion of my project. HDR rendering is a technique that is used for mapping more detail from the calculations of a scene to the scene's final image. HDR rendering allows the scene to become more vibrant and preserve details that otherwise before would have been clamped.

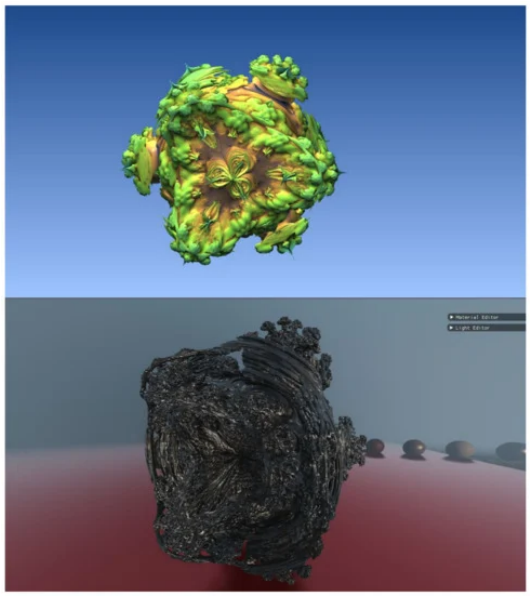
3 Challenge:

The project ahead will hold many challenges due to the nature of the techniques I will implement. The biggest problem with the raymarching will likely be learning how to utilize the compute shader in a way that evenly distributes the calculations for the scene across my GPU for maximum performance. Another challenge with the raymarching technique will be to identify the bottlenecks of the implementation and getting all the parameters just right to produce a vibrant image with good detail at a moderate frame rate. Specifically, the marching algorithm itself (Sphere Tracing) will need to be implemented using a specific algorithm as well as configuring the number of iterations I allow each ray to move through the scene. Of course understanding the Mandelbulb SDF, its limitations, and all the final color calculations of Orbit trap coloring, Blinn-Phong, and shadow rays will take some trial and error too. This all comes down to properly exploiting the power of a compute shader. I will need to setup a compute shader pass and a storage image so that I can render my results to the storage image that will then be processed again in the fragment shader. . Once I am able to overcome the challenges behind employing the compute shader, I will need to overcome challenges related to HDR rendering which start at the end of the compute shader when I write out to the storage image in the specified 16-bit float format. This will involve many details such as creating the storage image, managing its memory, ensuring I can use it with a new descriptor layout, and managing layout transitions. This all needs to coincide with the final tone mapping step where I will need to access the storage image in the fragment shader. I will also need to learn more about synchronization of the fragment shader reading the storage image and the compute shader writing to the storage image each frame using fences. To which finally before I am able to get my result I need to pick out a tone mapping compression method. Hence, it will really be the culmination of all these smaller challenges concatenated into one

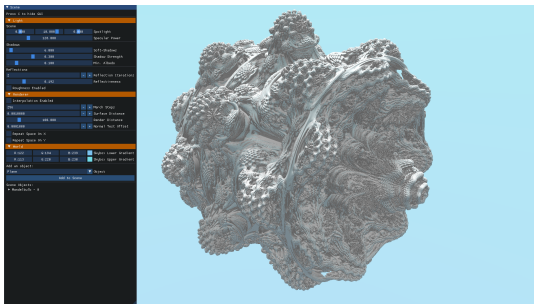
project and trying to get all the pieces parts working together into one cohesive Vulkan program that will be my biggest challenge.

4 Existing Work:

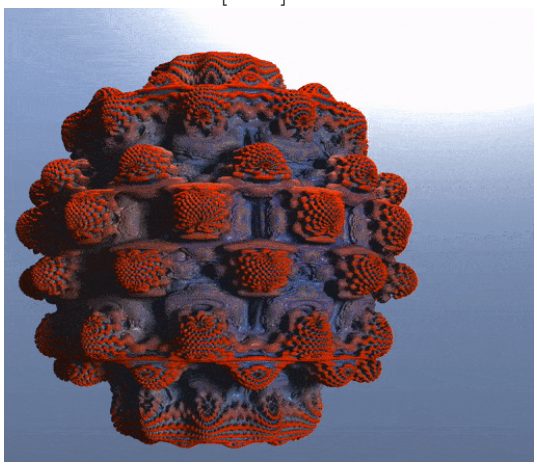
There is a lot of existing work and samples of raymarching, and I was able to find lots of resources. I found a professional paper, an article and even a whole website dedicated to raymarching, signed distance functions, and modifications to those functions. I have found many GitHub repositories with code examples of raymarching and even specific repositories where fractal rendering is a focus. I have also found repositories from professional sources showcasing example code for HDR rendering in Vulkan specifically. The Vulkan tutorial goes over the basics of setting up a compute shader pass, and I have found a couple of videos of YouTube that exceptionally discuss and showcase the process of raymarching and fractal rendering. Below are some sample images of similar projects to mine that I found with the sources mentioned above and given below:



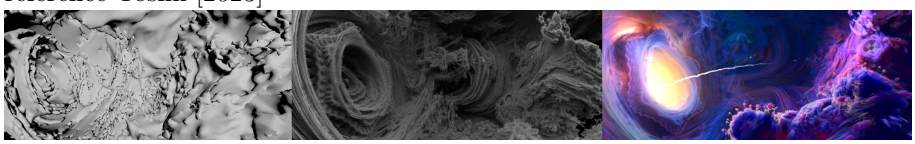
reference Hadji-Kyriacou and Arandjelović [2021a]



reference Schmidt [2020]



reference Yoshii [2023]



reference Animation [2014]

References

D. Animation. Into the portal: Big hero 6. 2014. URL `https://disneyanimation.com/publications/big-hero-6-into-the-portal/`.

CelticSpwn. Cs184 final project. `https://celticspwn.github.io/CS184FinalProject/final.html`, 2019. UC Berkeley student project.

K. Group. Vulkan hdr sample. `https://github.com/KhronosGroup/Vulkan-Samples/tree/main/samples/api/hdr`, 2023.

A. Hadji-Kyriacou and O. Arandjelović. Raymarching distance fields with

cuda. *Electronics*, 10(22), 2021a. ISSN 2079-9292. doi: 10.3390/electronics10222730. URL https://www.mdpi.com/2079-9292/10/22/2730.

A. Hadji-Kyriacou and O. Arandjelović. Raymarching distance fields with cuda. *Electronics*, 10(22), 2021b. doi: 10.3390/electronics10222730. URL https://www.mdpi.com/2079-9292/10/22/2730.

A. Overvoorde. *Vulkan Tutorial*. 2016. URL https://vulkan-tutorial.com/.

I. Quilez. Distance functions, 2019. URL https://iquilezles.org/articles/distfunctions.

I. Quilez et al. Shadertoy. https://www.shadertoy.com/, 2013.

T. Schmidt. 3d raymarching, 2020. URL https://timm-schmidt.com/3d-raymarching/.

S. Willems. Hdr example. https://github.com/SaschaWillems/Vulkan/blob/master/examples/hdr/hdr.cpp, 2024.

K. Yoshii. Raymarcher repository. https://github.com/KentaYoshii/Raymarcher, 2023. GitHub repository.

YouTube. Raymarching video tutorial (playlist). https://www.youtube.com/watch?v=Cp5WWtMoeKg&list=PL588FWrJlcvIcjhnkMWmLRD9JA-_AGI0U&index=3, 2020a.

YouTube. Raymarching tutorial. https://www.youtube.com/watch?v=khblXafu7iA&t=1777s, 2020b.